Sutra Code Working Overview

Sutra reads an optical pulse signal and body movement, calculates heart rate and heart rate variability, and compares the results with reference values. A sustained RMSSD drop can start a two-stage vibration intervention. Breathing rate and device status are also displayed.

1 Startup and sensor readings

On power-up, the ESP32-S3 restores saved baseline history, checks the motors, starts the MAX30101 pulse sensor and BMI270 motion sensor, and starts the background PDR and BLE services. The main loop then repeatedly reads sensors and updates the calculations.

- MAX30101 supplies the green optical pulse signal. Contact must be detected and settle for about 1.5 seconds before normal processing.
- BMI270 supplies acceleration and rotation data. The code classifies movement as LOW, MODERATE or HIGH.
- The optical sensor is configured at 400 Hz. The original beat detector uses fixed windows intended for the approximately 100 Hz processing stream noted in the code. BMI270 reads are scheduled every 10 ms. Actual delivered rates require device measurement.

2 Heart rate and beat intervals

The original LMS path uses motion information to reduce movement noise in the optical signal. An Elgendi detector finds pulse regions and selects a peak. Accepted peak timings give the inter-beat interval or IBI. The code smooths the resulting heart rate for display.

A separate HIGH-motion processor cleans the signal using all three acceleration axes and its own Elgendi detector. It tracks a separate rhythm for walking tests. With the current switches enabled, this processor supplies displayed HR during HIGH motion.

In this upload, displayed LMS IBI still comes from the original path. HIGH-motion HR and displayed LMS IBI can therefore come from different processors. HIGH-motion intervals do not feed RMSSD, PDR or P1.

3 Heart rate variability

Accepted individual IBIs pass additional artifact checks before entering a 60-second rolling RMSSD window. RMSSD describes how much the interval changes from one beat to the next. HR can remain available even when intervals are unsuitable for RMSSD.

4 Reference values and deviation

Sutra compares current RMSSD with three references. These are separate from the 30-second P1 trigger counters.

- Session baseline: collects 20 qualifying readings in five-second blocks, giving 100 qualifying seconds. The median then stays fixed until the next startup.
- Short reference: adapts using readings from the latest five minutes. It needs at least 12 qualifying readings, equivalent to 60 seconds.
- Personal baseline: stores the latest seven completed startup session medians in flash and uses their median. These represent startup sessions, rather than seven calendar days.

DEV is the percentage drop below a reference: (reference RMSSD − current RMSSD) ÷ reference RMSSD × 100. For example, a reference of 100 ms and current RMSSD of 85 ms give a 15% drop.

5 P1 trigger and vibration sequence

The current code uses a 15% RMSSD drop held for 30 seconds. Each available reference has its own hold timer. One completed timer is enough to trigger P1.

MODERATE motion allows P1 counting to continue. HIGH motion resets pending holds. After recovery has been armed, HIGH motion also starts a requirement for 60 seconds of LOW motion before normal reference and trigger operation resumes; MODERATE pauses that recovery countdown.

When P1 triggers, the device runs a five-second flutter across the three motors. After the flutter stops successfully, it waits five seconds, then runs one 12-second breathing pattern. There is no independent P2 detection. Once armed, the follow-up is independent of later DEV or motion changes, although a haptic failure can cancel it.

After successful follow-up completion, the episode and hold timers reset immediately. If DEV is still high, a fresh 30-second hold is needed. The total trigger count remains cumulative.

6 Breathing rate and output

PDR estimates breaths per minute from breathing-related changes in beat intervals, called RIFV. It uses a 45-second window with a minimum coverage setting of 30%. Coverage alone does not guarantee a value; other validity conditions still apply. RIAV, based on pulse amplitude, remains available for diagnostics. PDR does not trigger or block P1.

Serial provides detailed status. BLE sends selected fields about once per second, including HR, IBI, personal baseline, DEV, ShortDEV, calibration progress, P1 counters, motion and PDR. In the selected BLE message, DEV is calculated against the session baseline. Background workers handle PDR and BLE while the main loop continues sensing.

7 Main files and their roles

The following files contain the main working logic. Paired headers define the settings, data structures and interfaces used by their source files.

File or folder	Purpose
src/main.cpp	Runs startup, sensor reads, motion classification, original LMS and Elgendi processing, RMSSD, P1 integration and output snapshots.
src/DigitalFilter.cpp and .h	Provide reusable digital filters for signal conditioning.
lib/BeatRecovery/BeatRecovery.h	Helps the original beat path reacquire a consistent rhythm after stale readings.
lib/HighMotionProcessor/	Contains the separate motion cancellation, beat detection and HR tracking used for HIGH-motion display.
lib/RmssdReferences/	Manages the adapting short reference, deviation comparisons, P1 holds, follow-up state and recovery timing.
lib/PersonalBaseline/	Collects the fixed session baseline and manages the seven saved session values.
src/NvsBaselineStore.h	Reads and writes baseline history in ESP32 flash, with integrity checks and fallback records.
lib/PdrObserver/	Estimates breathing rate from accepted beat history and tracks coverage and validity.
src/PdrService.cpp and .h	Pass beat and context events to the background PDR worker and return its latest result.
lib/HapticPatterns/	Generates motor patterns and tracks playback state without stopping the sensor loop.
lib/HapticArray/ and src/HapticWireBus.h	Communicate with three DA7280 drivers through TCA9548A channels 0, 1 and 2.
lib/TelemetryReport/	Defines shared snapshots and formats detailed reporting groups.
src/BleTelemetry.cpp and .h	Runs BLE advertising, connection handling and selected telemetry transmission.
platformio.ini and scripts/erase_on_upload.py	Set the board and dependencies. The upload script clears flash on firmware upload, including saved baselines.
test/ and platformio-test.ini	Provide host-side checks for algorithm, baseline, haptic and telemetry behaviour.
docs/	Contains supporting explanations and PDF flowchart links. Some descriptions differ from the current code.

This guide describes the uploaded implementation. It does not establish sensor accuracy or physical motor performance. The firmware was not changed.

8 Programming and flashing procedure and tools

Use the PlatformIO project supplied with Sutra to build and upload the firmware to the Seeed Studio XIAO ESP32S3.

Tools required

- Computer with Visual Studio Code and the PlatformIO extension, or PlatformIO command line tools already installed.
- Complete Sutra project folder, including platformio.ini, src, lib and scripts.
- USB data cable and the Sutra board. Internet access is needed for the first download of build tools and dependencies.

Procedure

1. Connect the board to the computer using a USB data cable. Open the Sutra project folder in PlatformIO and allow its dependencies to finish installing.
2. Select the seeed_xiao_esp32s3 environment from platformio.ini. Identify the board's current serial port; the port number may change between computers or boot modes.
3. Build the firmware using PlatformIO Build, or run: pio run -e seeed_xiao_esp32s3. Resolve any build errors before uploading.
4. Close any serial application using the board's port. Select PlatformIO Upload, or run: pio run -e seeed_xiao_esp32s3 -t upload. If automatic port selection fails, add --upload-port followed by the actual port.
5. After upload finishes, open Serial Monitor at 115200 baud. If startup messages are not visible, reset the board. Check the pulse sensor, motion sensor and haptic initialization messages, and confirm that regular telemetry follows.
6. Keep steady sensor contact and check calibration progress. Then connect over BLE using the next procedure. Record the firmware version or source revision used for the test.

Flash storage: this project runs an erase-on-upload script. Every firmware upload clears saved personal-baseline sessions and other flash data. Ordinary resets and power cycles retain saved baseline history.

If the board is not detected, first check the USB data cable and selected port. If needed, enter the board's bootloader using its BOOT and RESET controls, then retry the upload. Check startup faults before continuing a test.

9 BLE test procedure

This test checks discovery, live telemetry and reconnection. It does not establish the accuracy of the physiological readings.

Tools required

Use a phone or computer with Bluetooth Low Energy support and a BLE test application that can browse custom services, enable notifications and display received bytes as text. Keep Serial Monitor open for comparison if a USB connection is available.

Connect and receive data

1. Power on Sutra. Enable Bluetooth and grant the BLE application any Bluetooth or scan permissions it requests. Start a nearby-device scan and select the device named Sutra.
2. Connect and open its custom service. Confirm the service UUID shown below.
3. Find the notification characteristic listed below and enable notifications. Use UTF-8 or text view. Received text may be split across multiple BLE packets, so view the assembled stream when the application supports it.
4. Confirm that device uptime and telemetry update about once per second. Look for LMS_HR, LMS_IBI, PersonalBaseline, DEV, ShortDEV, Cal100, Session30, Short30, P1, PDR, PDRStatus and Motion. Some values may initially show -- while data or baselines are unavailable.

Service UUID: 7d2a0000-8f5a-4b10-9b6d-6f7574726100

Notification UUID: 7d2a01ff-8f5a-4b10-9b6d-6f7574726100

Functional checks

- With steady contact and LOW motion, verify that readings appear as the device acquires usable data and calibration progresses.
- Move the device and confirm Motion changes when the classifier detects movement. Compare BLE fields with Serial at matching device uptimes. HIGH-motion HR may use the separate processor, while LMS_IBI remains from the original path.
- Briefly remove sensor contact and confirm unavailable values or status changes appear as contact and freshness rules take effect. Restore contact and allow the device to reacquire readings.
- Disconnect, scan again, reconnect and re-enable notifications. Confirm that live updates resume. Reconnection should not restart device uptime unless the device itself reset.
- If P1 occurs during a test, compare the displayed P1 status with the felt flutter and follow-up. Record missing vibration, timing differences or driver faults separately from BLE reception.

A successful communication test means Sutra is discoverable, notifications arrive, the expected fields update, and reconnection works. If data is missing, check the selected characteristic and notification subscription, then compare the Serial BLE status. Save the received log and firmware revision with the test result.